ICT232 2024S2 Exam

Feel free to contact me at maths_and_programming@outlook.sg or @geodome on Telegram if you need tuition.

I provide undergraduate level tuition for the following SUSS units:

- ENG301 Microprocessor Progtramming
- ICT133 Structured Programming
- ICT162 Object Oriented Programming
- ICT325 Data Structures and Algorithms
- ICT323 Introduction to Parallel Computing
- ICT325 Design and Analysis of Algorithms
- ICT330 Database Management Systems
- MTD215 Application of C++ in Multimedia
- MTH105 Fundamentals of Mathematics
- MTH109 Calculus
- MTH207 Linear Algebra
- MTH210 Fundamentals of Probability
- MTH212 Statistical Analysis
- MTH240 Engineering Mathematics 1
- MTH316 Multivariable Calculus
- MTH321 Engineering Mathematics 2
- MTH220 Statistical Methods and Inference
- MTH355 Basic Mathematical Optimisation
- MTH367 Network Optimisation and Modelling

Donaldson Tan

SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

ICT232

Examination - July Semester 2024

Data Structures and Algorithms II

Friday, 22 November 2024 **04:00 pm - 06:00 pm**

Time allowed: 2 hours last 15min cannot leave

INSTRUCTIONS TO STUDENTS:

1. This examination contains 7 questions and comprises **6** printed pages (including cover page).
2. You must answer 7 questions.
3. All answers must be written in the answer book.
4. Ensure all answers are clear and concise.
5. For programming or algorithmic questions, you may write pseudocode if not specified otherwise.
6. For multiple choice questions, select the most appropriate answer. There is only one correct answer per question. If you are asked to describe why you have selected a specific answer, provide concise and accurate theoretical/technical descriptions supporting your decision.
7. You may include any necessary diagrams to elaborate your answer.
8. This is **Closed-book** examination.

$\frac{120}{7} = 17$ min per question.

At the end of the examination.
Please ensure that you have written your examination number on each answer book used.

Failure to do so will mean that your work cannot be identified.
If you have used more than one answer book, please tie them together with the string provided.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

ICT232 Copyright © 2024 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

Question 1a

Which of the following operations has the time complexity exceeding $O(\log n)$ when performed on a priority queue implemented with a binary heap?

A. Insert - Adds an element to the priority queue.
B. Find-Min - Retrieves the smallest element without removing it.
C. Remove-Min - Removes and returns the smallest element.
D. Find any element - Searches for a specific element within the priority queue.

(2 marks)

Question 1b

Explain why the operation selected in Q1(a) exceeds $O(\log n)$. Please include a brief description of how a binary heap works in your explanation.

(6 marks)

Question 1c

What is the time complexity of the operation identified as inefficient in Q1(a)?

(2 marks)

Question 2

Question 2a

What is the time complexity of searching for an element in a standard skip list?

(2 marks)

Question 2b

Explain why this complexity is expected, focusing on the structure and design of skip lists. First, describe the structure of a skip list, and then explain how its design contributes to this performance.

(8 marks)

Question 3

Develop pseudocode for inserting a new element into a Binary Search Tree (BST) using a recursive method. If an element with the same value already exists, update the node with the new value.

(15 marks)

Question 4

Consider the Quick-sort algorithm. Analyze and describe how the choice of pivot affects the algorithm's performance. Discuss what happens when:

(a) The pivot is always the smallest or the largest element in the sub-array being sorted.

(b) The pivot is chosen randomly from the sub-array.

(c) The pivot is the median value (or an approximate median like median-of-three) of the elements in the sub-array.

Include in your answer the implications of each pivot choice on the time complexities of the algorithm.

(15 marks)

4x17
40+28= 68

Question 5

Question 5a

Describe how Dijkstra's Algorithm is used to find the shortest path from a source vertex to a destination vertex.

(5 marks)

Question 5b

Analyse the graph shown in Figure Q5 to identify the shortest path from vertex A to vertex E using Dijkstra's Algorithm. Detail each step taken to determine the total weight of the shortest path using the table format given below.

ABCDED

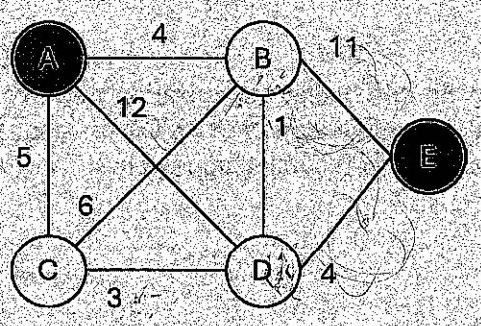

Figure Q5

Please use this table format to illustrate each step.

v_i	Status	$D[v_i]$	Previous Vertex
A			
B			
C			
D			
E			

(10 marks)

Question 5c

Explain in detail why Dijkstra's Algorithm may fail to provide the correct results when there are negative weight edges in the graph. Ensure your explanation includes how the algorithm assumes edge weights and its reliance on these assumptions.

(5 marks)

Question 6

Develop a memoized dynamic programming version of the Fibonacci sequence calculation and compare its computational efficiency to the traditional recursive method. Follow detailed instructions shown in Q6(a), Q6(b), and Q6(c).

Question 6a

Describe the Recursive Method. Provide a concise description of the traditional recursive method for calculating the Fibonacci sequence. Explain how the Fibonacci number is computed using recursion.

(6 marks)

Question 6b

Pseudocode for Memoization. Develop detailed pseudocode for the Fibonacci sequence using the memoization technique. Ensure that your pseudocode clearly illustrates how results of computations are stored and reused.

(6 marks)

Question 6c

Efficiency Comparison. Explain how using memoization improves computational efficiency compared to the traditional recursive approach. Clearly identify and compare the time complexity of both the memoized and recursive approaches.

(6 marks)

Question 7

Question 7a

What property must be true for all nodes in a binary search tree (BST)?

A. The left child's value must be strictly greater than the parent node's value.
B. The right child's value must be strictly less than the parent node's value.
C. The left child's value must be less than, and the right child's value greater than, the parent node's value.
D. Each node can have no more than two children .

(2 marks)

Question 7b

Which of the following sorting algorithms typically exhibits the best average-case performance for sorting operations?

A. Quick sort
B. Heap sort
C. Bubble sort
D. Insertion sort

(2 marks)

Question 7c

What is the primary objective of a hash function in a hash table?

A. To encrypt data before storing it in the table.
B. To uniformly distribute key-value pairs across the hash table to minimize collisions.
C. To compress the keys into a smaller format for easier storage.
D. To sequentially order key-value pairs for faster retrieval.

(2 marks)

Question 7d

In a directed graph, what does an edge from node A to node B represent?

A. A bidirectional relationship between A and B.
B. A relationship where A depends on B.
C. A relationship where B can reach A but not vice versa.
D. A relationship where A can reach B.

(2 marks)

Question 7e

What is a key advantage of the Knuth-Morris-Pratt (KMP) pattern-matching algorithm over the brute force approach?

A. KMP algorithm has a worse-case time complexity of O(n^2).
B. KMP algorithm processes the text from right to left.
C. KMP algorithm utilizes partial match information to avoid needless re-examination of text.
D. KMP algorithm requires additional space for storing the pattern.

(2 marks)

Question 7f

Which statement best describes the difference between linear probing, quadratic probing, and double hashing as methods for collision resolution in hash tables?

A. Linear probing searches for the next available slot one step at a time, quadratic probing searches at increasingly larger intervals, and double hashing uses a second hash function to determine the interval size.
B. Linear probing and quadratic probing use a secondary hash function to determine the next slot, while double hashing increments the slot index by a fixed constant.
C. Linear probing increments the slot index by a fixed constant, quadratic probing uses the original hash value for all attempts, and double hashing recalculates a new hash for each attempt.
D. All three methods recalculate a new hash for each attempt, but they differ in the hash functions used.

(2 marks)

----- END OF PAPER -----

ICT232 Copyright © 2024 Singapore University of Social Sciences (SUSS)

ICT232 2024S2 Solution

1(a)	Insertion involves up-heap, so time complexity is O(log n) Find-Min is just peeking the min element at the root of the binary heap, so it is O(1) Remove-Min involves down-heap, so time complexity is O(log n) Find any element requires inspecting every element, so it is O(n) D is the option with time complexity exceeding O(log n).
1(b)	The priority queue is implemented as a binary heap using priority as the sort key. The binary heap is in fact a complete binary tree with the property that the parent node's priority is higher than the child node's priority. Hence, the heap property cannot be used for searching for an element since the element itself is not the sort key. Instead, during the search element process, every element must be inspected, hence its time complexity is O(n) instead of O(log n).
1(c)	The time complexity is O(n)

2(a)	O(log n)
2(b)	A skip list a multi-layered linked list with the lowest layer linked list containing every element. The probability of an element present in each layer n is given by $(1/2)^n$ where n is an integer. When an element is inserted in the skip list, it is first added to the lowest layer [Layer 0]. Then randomisation is used to decide whether to add the element to the next level. If the next level is not added, then no additional layers are added. The search process starts at the top most node of the multi-layered linked list, so it consists of vertical traversal and horizontal traversal. The vertical traversal is contained by the height of the linked list which is given as O(log n) while the expected number of forward traversal is just 2 steps hence the time complexity of the search process is O(log n)

ICT232 2024S2 Solution

3	name:	insert(BST, e)
	input:	BST (the binary search tree), Element e
	output	An updated Binary Search Tree containing the element e

	procedure:	if BST is empty then
		  BST.root = new node with element e
		else
		  recursiveInsert(BST.root, e)

	name:	recursiveInsert(current, e)
	input:	current - a node in the BST e - the element to be inserted in the BST
	output	An updated Binary Search Tree containing the element e

	procedure:	if current.element == element then
		  current.element = element
		else if element > current.element then
		  if current.right is None then
		  current.right = new node with element e
		  else
		  recursiveInsert(current.right, e)
		else
		  if current.left is None then
		  current.left = new node with element e
		  else
		  recursiveInsert(current.left, e)

4(a)	When the pivot is always the smallest or biggest element of the subarray, quick sort effectively becomes a selection sort with performance of $O(n^2)$
4(b)	A good pivot is one that falls in the inter-quartile range of the subarray. Hence, a randomly chosen pivot has a 50-50 chance of becoming a good pivot or a bad pivot, giving rise to time complexity of expected O(n log n)
4(c)	When the pivot is median, then the subarray will be equally divided, giving the time complexity of O(n log n).

5(a)	Djikstra's algorithm is essentially a weighted Bread First Search algorithm that computes the shortest distance from a given source node to the other nodes. During each iteration, it uses the relaxation technique to update the distance from the source node to the current node if a shorter distance can be found.

ICT232 2024S2 Solution

5(b)	

Vi	Status	D[Vi]	Previous Vertex
A		0	NA
B		Infinity	
C		Infinity	
D		Infinity	
E		Infinity	

Vi	Status	D[Vi]	Previous Vertex
A	Current	0	NA
B		4	A
C		5	A
D		12	A
E		Infinity	

Vi	Status	D[Vi]	Previous Vertex
A	Visited	0	NA
B	Current	4	A
C		5	A
D		5	B
E		15	B

Vi	Status	D[Vi]	Previous Vertex
A	Visited	0	NA
B	Visited	4	A
C	Current	5	A
D		5	B
E		15	B

ICT232 2024S2 Solution

Vi	Status	D[Vi]	Previous Vertex
A	Visited	0	NA
B	Visited	4	A
C	Visited	5	A
D	Current	5	B
E		9	D

Vi	Status	D[Vi]	Previous Vertex
A	Visited	0	NA
B	Visited	4	A
C	Visited	5	A
D	Visited	5	B
E	Current	9	D

Vi	Status	D[Vi]	Previous Vertex
A	Visited	0	NA
B	Visited	4	A
C	Visited	5	A
D	Visited	5	B
E	Visited	9	D

The shortest path is A->B->D->E of weight 9.

5(c)	The negative weight breaks the relaxation technique used in each iteration to determine whether or not to update the distance from the source node to the current node because it contradicts the assumption that adding an edge to a path would increase the distance from the source node.

ICT232 2024S2 Solution

6(a)	name:	fib(n)
	input:	n is an integer >= 0
	output	the n-th fibonacci number

	procedure:	if n == 0 or n == 1 then
		  return 1
		else
		  return fib(n-1) + fib(n-2)

6(b)	name:	fib(n, table)
	input:	n is an integer >= 0 table is a Map ADT
	output	the n-th fibonacci number

	procedure:	if n == 0 or n == 1then
		 return 1
		else
		  if n in table then
		  return table[n]
		  else
		  table[n] = fib(n-1, table) + fib(n-2, table)
		  return table[n]

6(c)	The time complexity of the traditional recursive approach is $O(2^n)$ because each call makes 2 recursion calls to compute the previous fibonacci numbers. However, the time complexity of the memoized approach is O(n) as each previous fibonacci number is only computed once and stored in the table for O(1) retrieval.	

7(a)	C, D are true
7(b)	Quick sort performs expected O(n log n) Heap sort performs O(n log n) Bubble sort performs $O(n^2)$ Insertion sort performs $O(n^2)$ Heap Sort is the best since its performance does not depend on randomisation
7(c)	B
7(d)	D